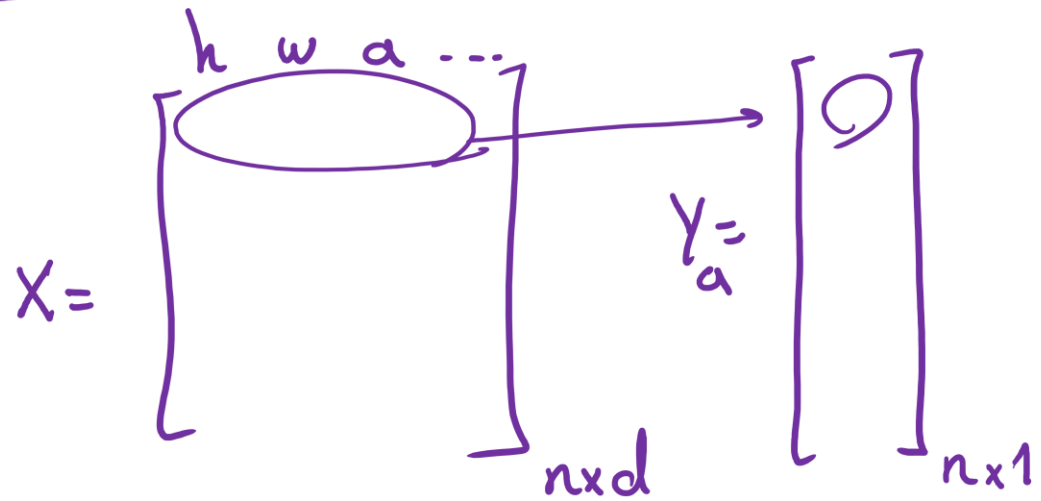


PCA $\leadsto$ dimensionality reduction $\leadsto$ unsupervised

Supervised learning dimensionality reduction



$$Z_{(1)} = \begin{bmatrix} h \\ \vdots \end{bmatrix}$$

$\downarrow$ NB
 $\downarrow$ acc = 0.65

$$Z_{(2)} = \begin{bmatrix} w \\ \vdots \end{bmatrix}$$

$\downarrow$ NB
 $\downarrow$ acc = 0.75

$$Z_{(3)} = \begin{bmatrix} a \\ \vdots \end{bmatrix} \dots$$

$\downarrow$ NB
 $\downarrow$ acc = 0.4

$$Z_{(1)} = \begin{bmatrix} w & h \\ \vdots & \vdots \end{bmatrix}$$

acc = 0.82

$$Z_{(2)} = \begin{bmatrix} w & a \\ \vdots & \vdots \end{bmatrix} \dots$$

0.71

Forward Feature Selection

Backward feature selection

$$X = \begin{bmatrix} \text{h} & \text{w} & \text{a} \dots \end{bmatrix} \quad Y = \begin{bmatrix} \end{bmatrix}$$

LDA

RF feature selection

Linear Regression

Mahdi Roozbahani
Georgia Tech

The Moment

© Yongqing Bao



Incoming ML HW3

You; after HW2

Outline

- Supervised Learning 
- Linear Regression
- Extension

1000

70% training

20% testing

10% validation

80% training

20% testing

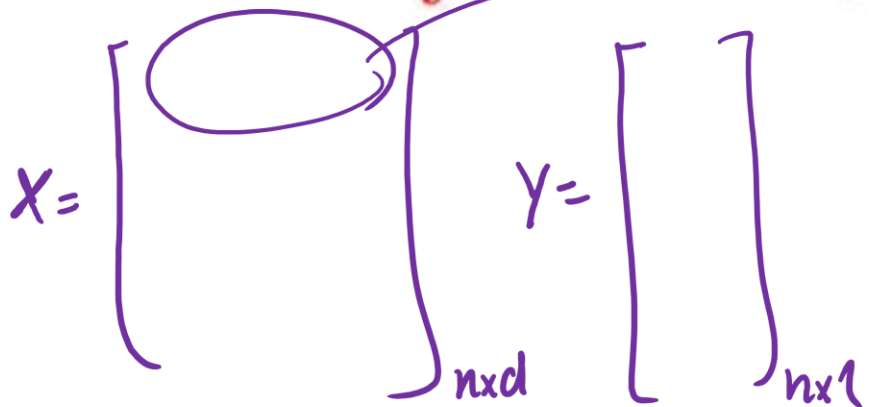
Supervised Learning: Overview

Functions $\mathcal{F}$

Training data

$$f: \mathcal{X} \rightarrow \mathcal{Y}$$

$$\{(x^{(i)}, y^{(i)}) \in \mathcal{X} \times \mathcal{Y}\}$$



LEARNING

$$\begin{aligned} &\text{find } \hat{f} \in \mathcal{F} \\ &\text{s.t. } y_a \approx f(x^{(i)}) \end{aligned}$$

y_a y_p



Learning machine

PREDICTION

$$\hat{y}_p = f(x^{(i)})$$

New data

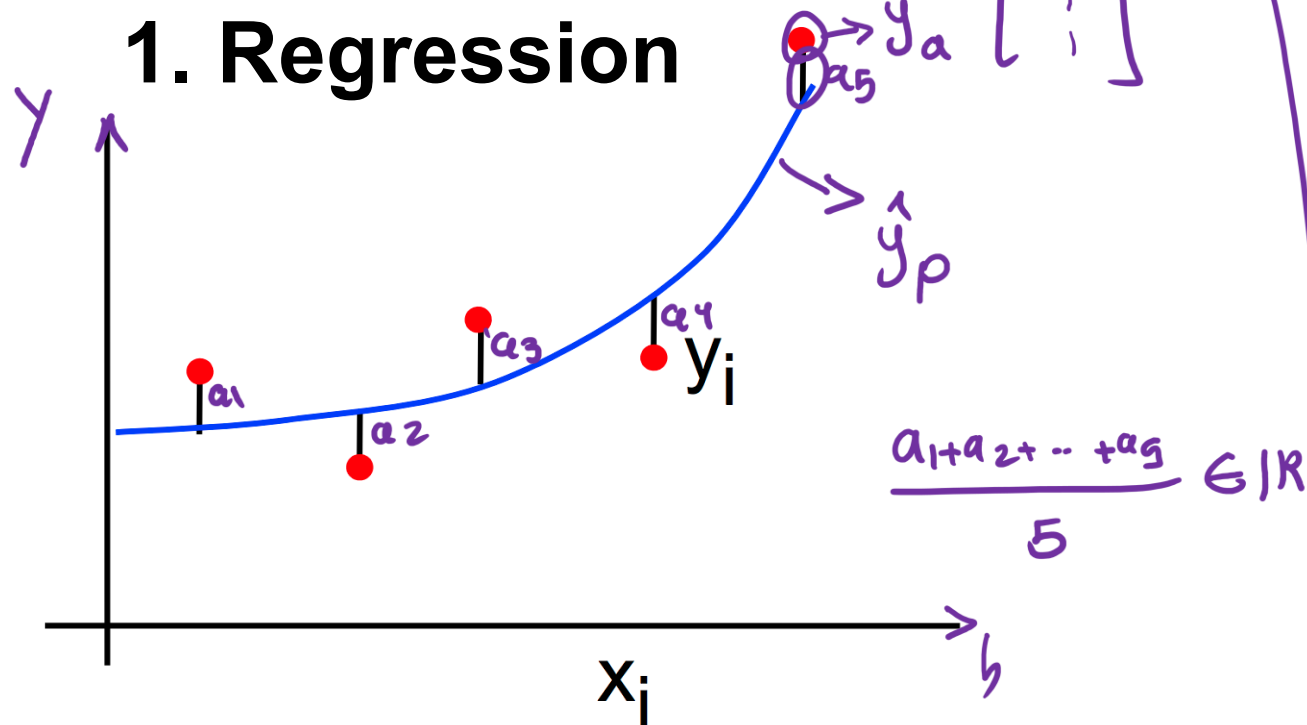
$$x$$

Supervised Learning: Two Types of Tasks

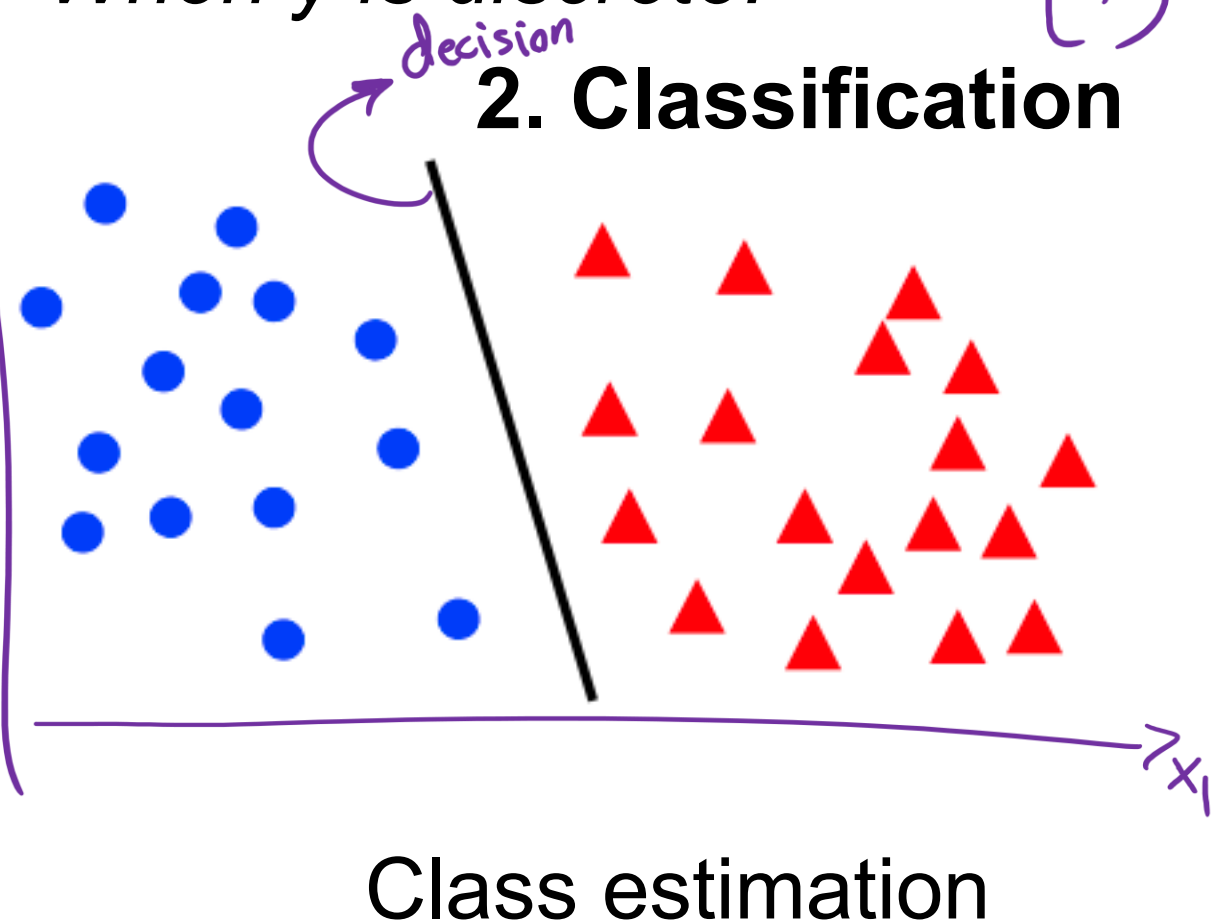
Given: training data $\{(x^{\{1\}}, y^{\{1\}}), (x^{\{2\}}, y^{\{2\}}), \dots, (x^{\{n\}}, y^{\{n\}})\}$

Learn: a function $f(\mathbf{x}) : y = f(\mathbf{x})$

When y is continuous:



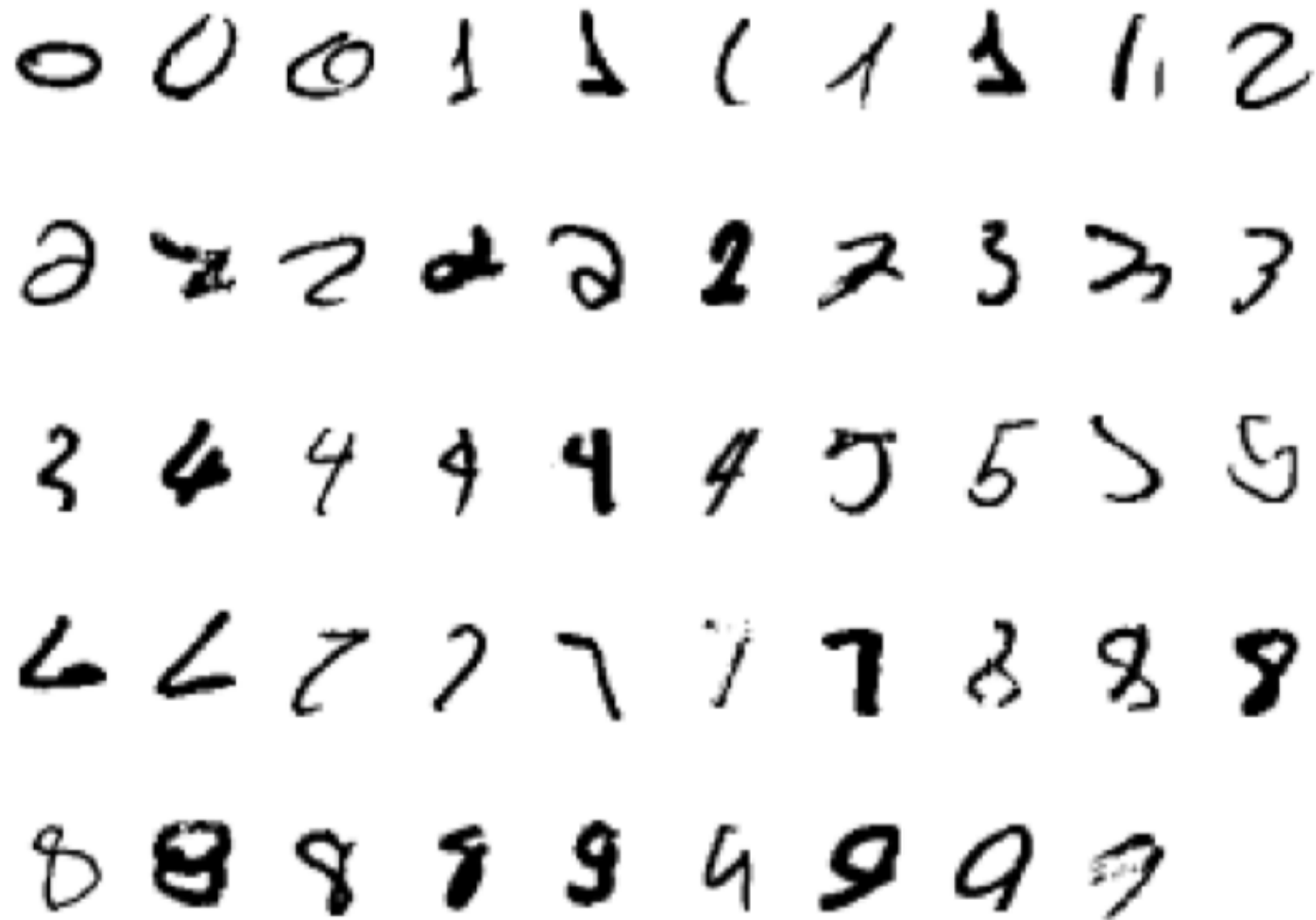
When y is discrete:



Classification Example 1: Handwritten digit recognition

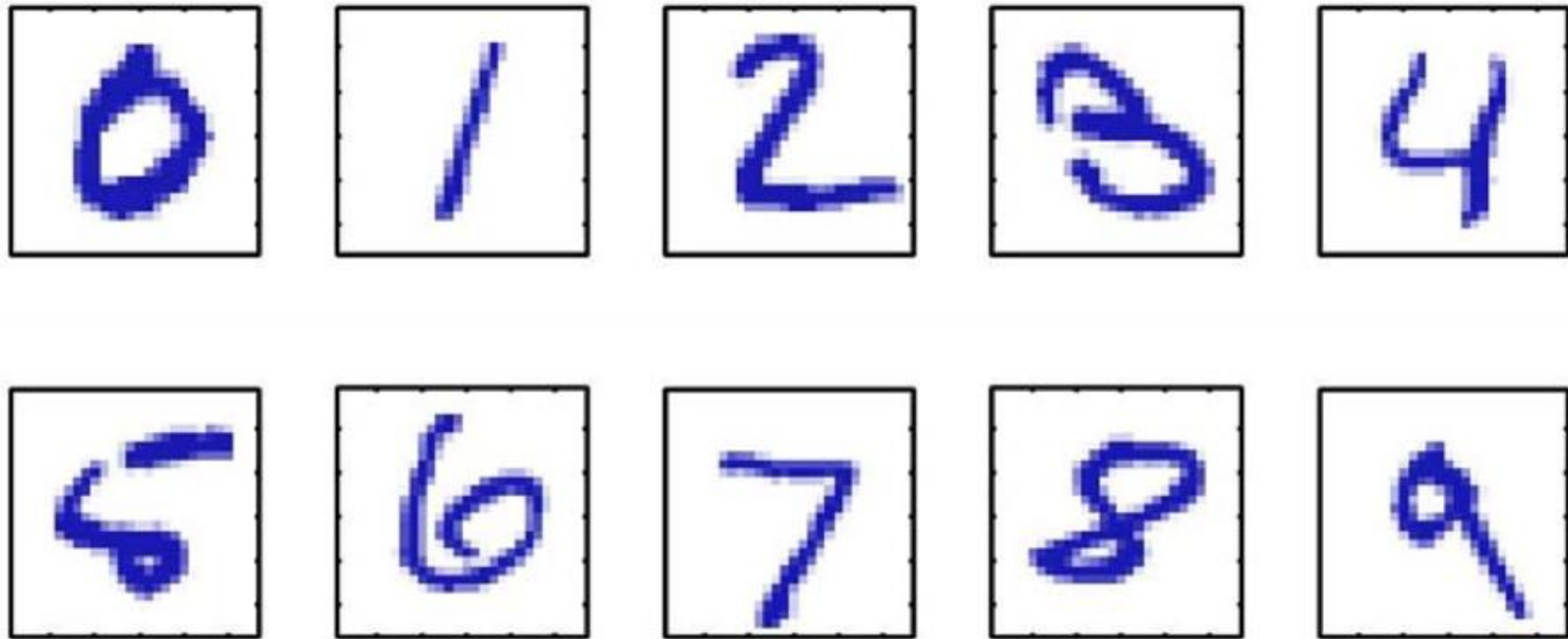
As a supervised classification problem

Start with training data, e.g. 6000 examples of each digit



- Can achieve testing error of 0.4%
- One of first commercial and widely used ML systems (for zip codes & checks)

Classification Example 1: Hand-Written Digit Recognition



Images are 28 x 28 pixels

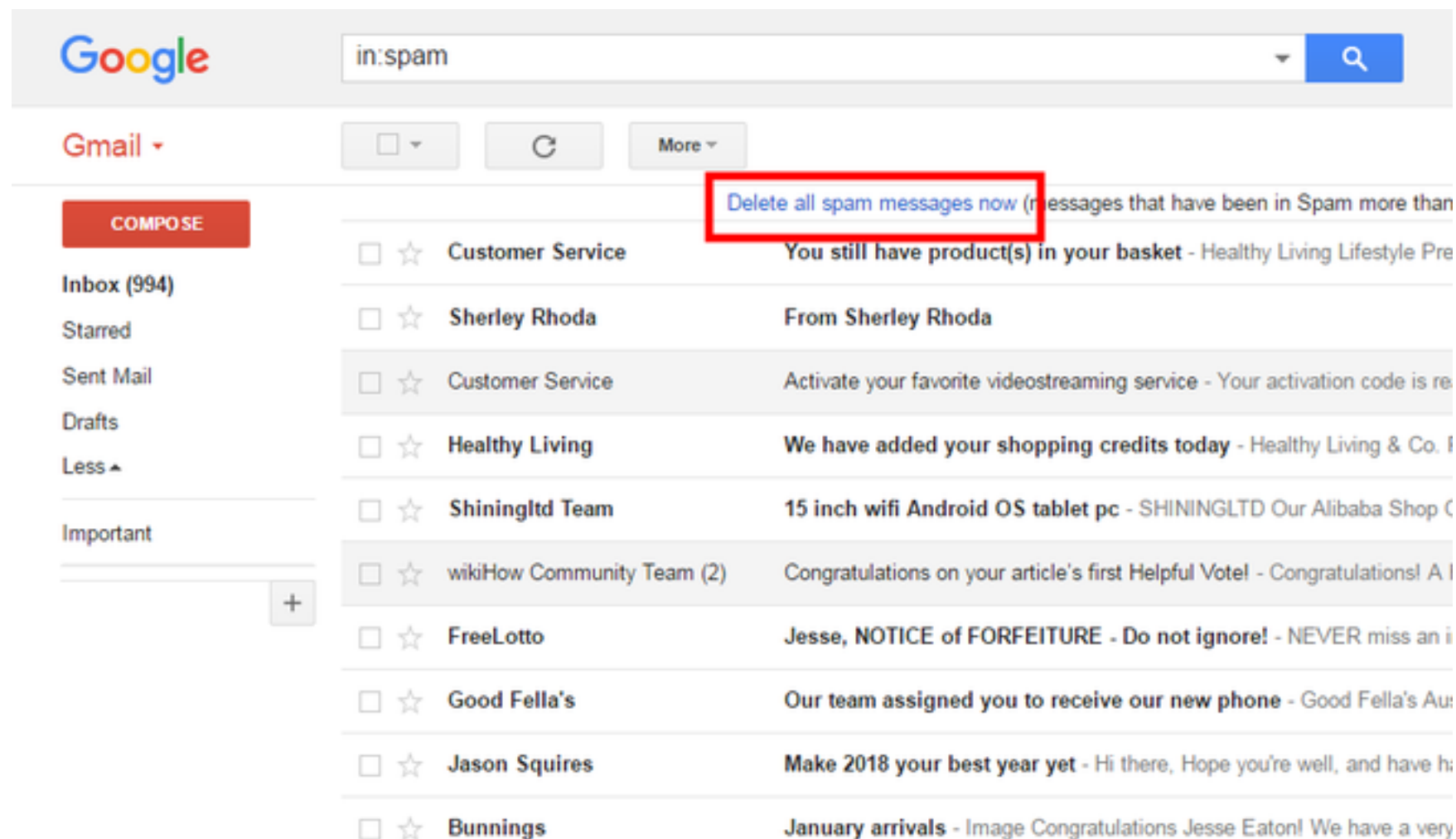
A classification problem

Represent input image as a vector $\mathbf{x} \in \mathbb{R}^{784}$

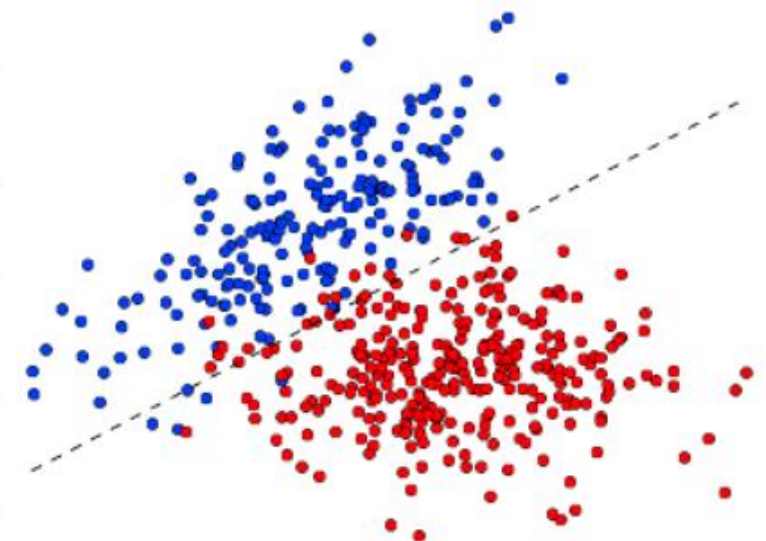
Learn a classifier $f(\mathbf{x})$ such that,

$$f : \mathbf{x} \rightarrow \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

Classification Example 2: Spam Detection



NOT SPAM



SPAM

A classification problem

- This is a classification problem
- Task is to classify email into spam/non-spam
- Data x_i is word count
- Requires a learning system as “enemy” keeps innovating

Regression Example 1: Apartment Rent Prediction

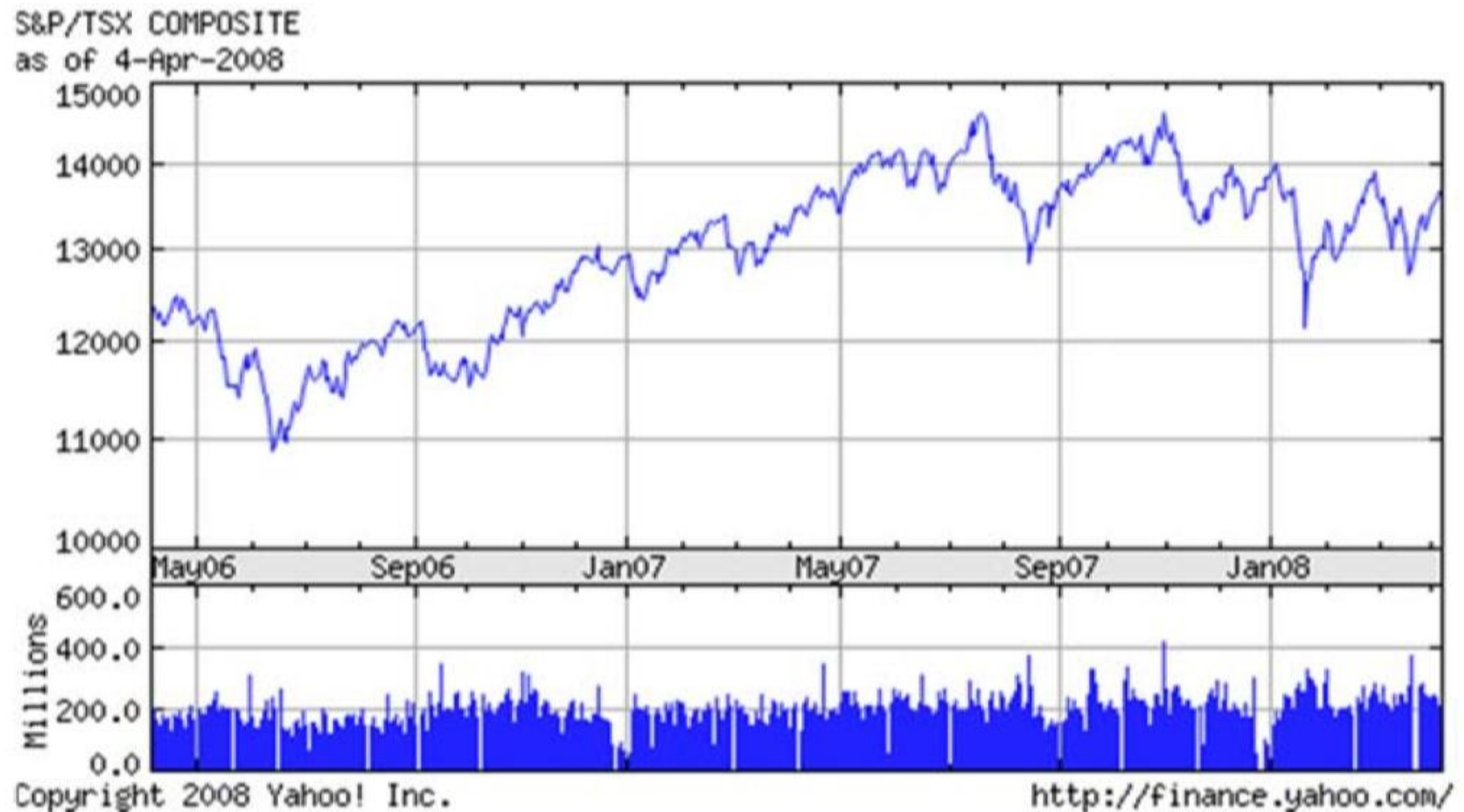
- Suppose you are to move to Atlanta
- And you want to find the **most reasonably priced** apartment satisfying your **needs**:

square-ft., # of bedroom, distance to campus ...

A regression problem

Living area (ft ²)	# bedroom	Rent (\$)
230	1	600
506	2	1000
433	2	1100
109	1	500
...		
150	1	?
270	1.5	?

Regression Example 2: Stock Price Prediction



- Task is to predict stock price at future date

A regression problem

$$\hat{y}_p = mx + b$$



$$\hat{y}_p = \underbrace{\Theta_0}_{\text{bias term}} + \underbrace{\Theta_1 x_1}_{\text{Feature one parameter}}$$

Feature one parameter

Features:

- Living area, distance to campus, # bedroom ...
- Denote as $x = (x_1, x_2, \dots, x_d)$

Target:

- Rent

- Denoted as y

$$X = [1, x_1, x_2, \dots, x_d]_{1 \times d+1}$$

$$\Theta = \begin{bmatrix} \Theta_0 \\ \Theta_1 \\ \vdots \\ \Theta_d \end{bmatrix}_{d+1 \times 1}$$

$$\hat{y}_p = X \cdot \Theta$$

$\rightarrow$ LCF

Linear combination of features

Training set:

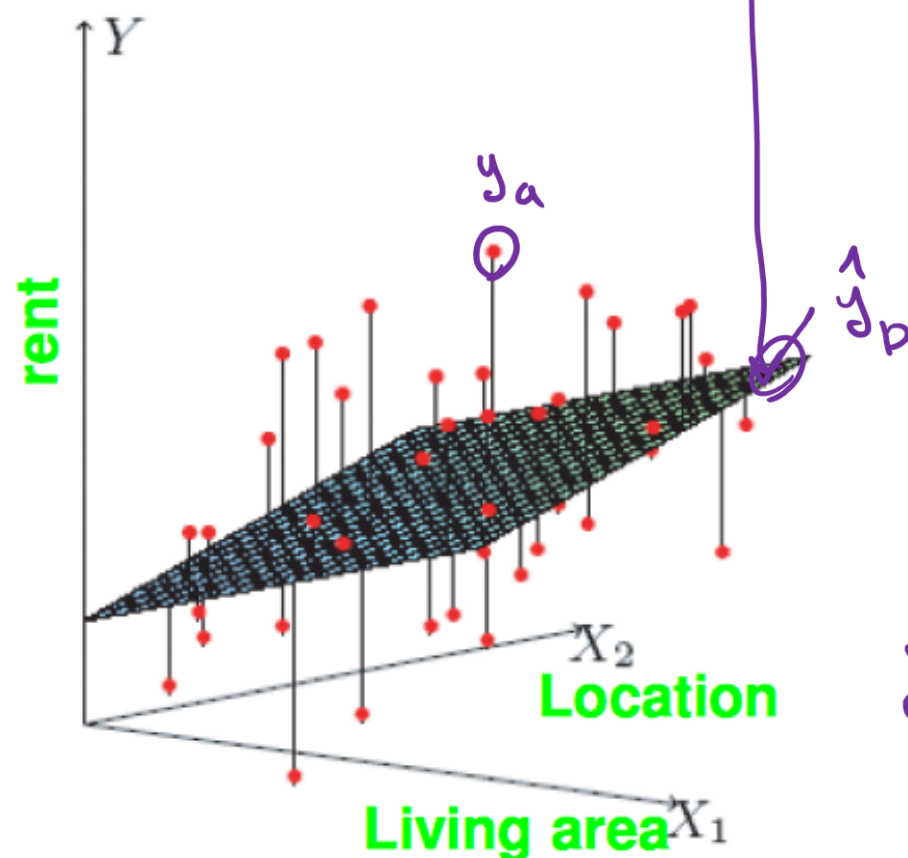
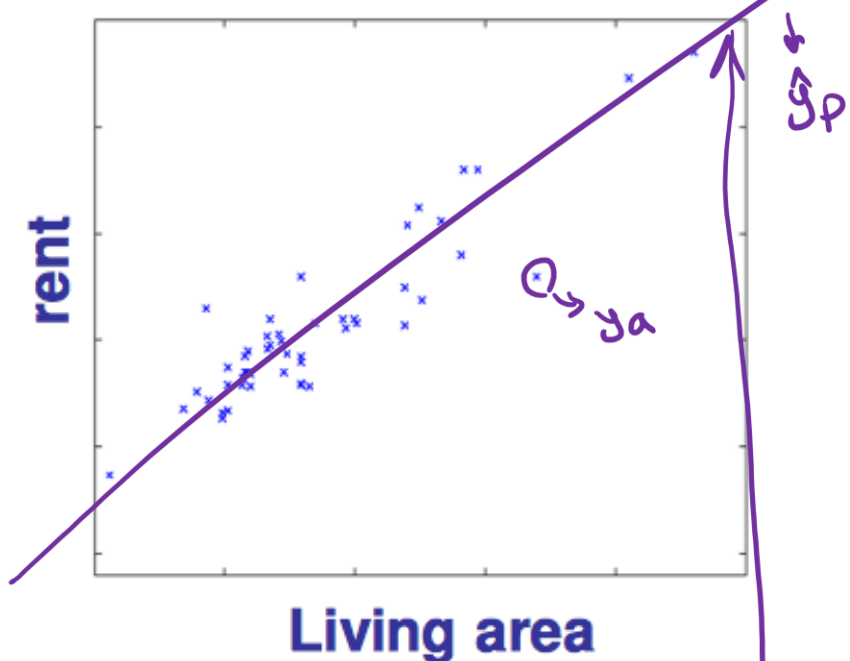
$$x = \{x^{(1)}, x^{(2)}, \dots, x^{(n)}\} \in R^d$$

$$y = \{y^{(1)}, y^{(2)}, \dots, y^{(n)}\}$$

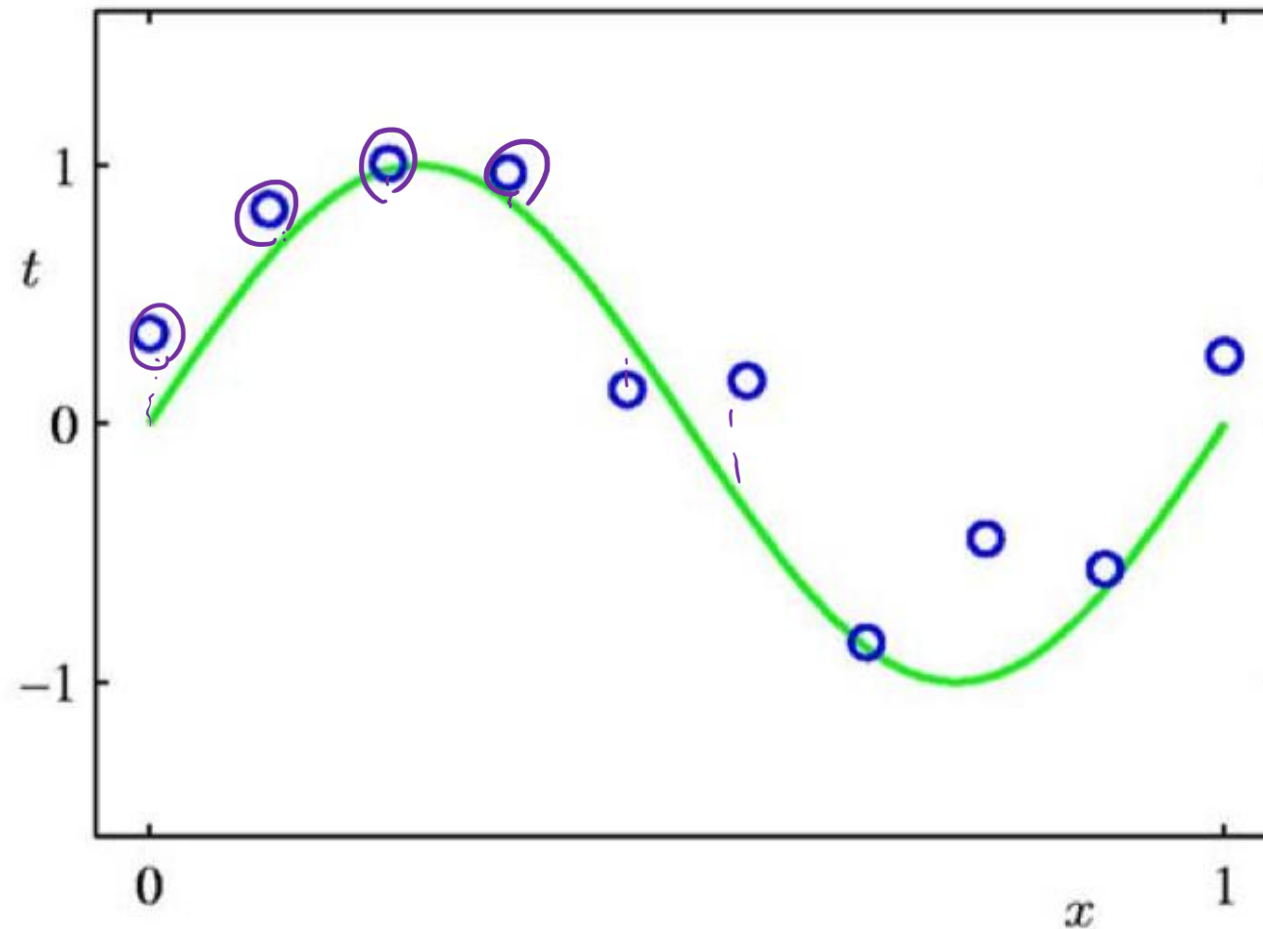
$$\hat{y}_p = \Theta_0 + \Theta_1 x_1 + \Theta_2 x_2$$

$$D = 2$$

$$\Theta = 3 = D + 1$$



Regression: Problem Setup



- Suppose we are given a training set of N observations
- Regression problem is to estimate $y(x)$ from this data

Outline

- Supervised Learning
- Linear Regression ← 
- Extension

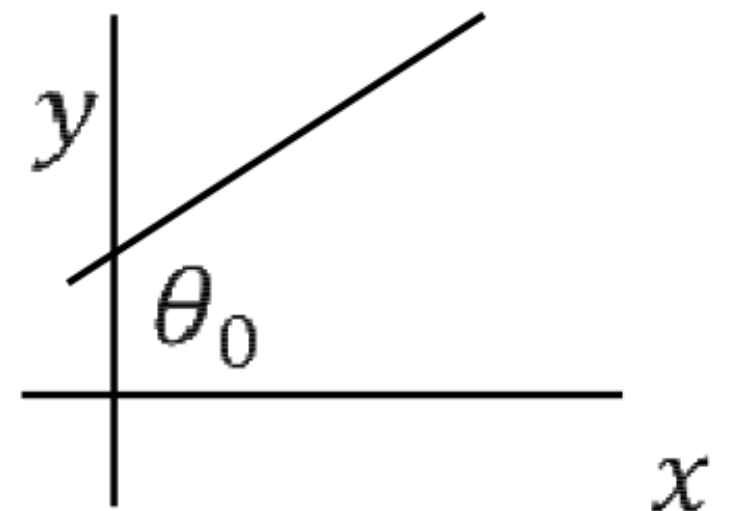
Linear Regression

- Assume y is a linear function of x (features) plus noise ϵ

$$y = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d + \epsilon = x\theta$$

- where ϵ is an error term of unmodeled effects or [random noise](#)
- Let $\theta = (\theta_0, \theta_1, \dots, \theta_d)^\top$, and augment data by one dimension

- Then $y = x\theta + \epsilon$



$E[\theta]$

$|y_a - \hat{y}_p|$ x^2

Least Mean Square Method

- Given n data points, find θ that minimizes the mean square error

Training $\hat{\theta} = \underset{\theta \in \mathbb{R}}{\operatorname{argmin}} L(\theta) = \frac{1}{n} \sum_{i=1}^N (y_a^{(i)} - \underbrace{x^{(i)} \theta}_{\hat{y}_p})^2 \leadsto \text{Convex}$

Handwritten notes: Loss, Error, $E(\theta)$, 1×1 , $1 \times d+1$, $d+1 \times 1$

- Our usual trick: set gradient to 0 and find parameter $\frac{\partial L(\theta)}{\partial \theta} = 0$

$y = X\theta$ $\theta = X^{-1}y$

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n} \sum_{i=1}^N (x^{(i)})^T (y^{(i)} - x^{(i)}\theta) = 0$$

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n} \sum_{i=1}^N (x^{(i)})^T y^{(i)} + \frac{2}{n} \sum_{i=1}^N (x^{(i)})^T x^{(i)} \theta = 0$$

Matrix form

$$x = \begin{bmatrix} 1 & x_1^{\{1\}} & \dots & x_d^{\{1\}} \\ 1 & x_1^{\{2\}} & \ddots & x_d^{\{2\}} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{\{n\}} & \dots & x_d^{\{n\}} \end{bmatrix} \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix}$$

$n \times (d + 1)$
 $n \times 1$
 $(d + 1) \times 1$

$$MSE(\theta) = \operatorname{argmin}_{\theta} L(\theta) = \frac{1}{n} (y - x\theta)^T (y - x\theta)$$

$$x\theta = \begin{bmatrix} \theta_0 + \theta_1 x_1^{\{1\}} + \theta_2 x_2^{\{1\}} + \dots + \theta_d x_d^{\{1\}} \\ \theta_0 + \theta_1 x_1^{\{2\}} + \theta_2 x_2^{\{2\}} + \dots + \theta_d x_d^{\{2\}} \\ \vdots \\ \theta_0 + \theta_1 x_1^{\{n\}} + \theta_2 x_2^{\{n\}} + \dots + \theta_d x_d^{\{n\}} \end{bmatrix} \quad \text{LCF for } x^{\{1\}} \quad x \cdot \theta$$

$n \times 1$

Matrix Version and Optimization

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n} \sum_{i=1}^N (x^{\{i\}})^T y^{\{i\}} + \frac{2}{n} \sum_{i=1}^N (x^{\{i\}})^T x^{\{i\}} \theta = 0$$

Let's rewrite it as:

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n} (x^{\{1\}}, \dots, x^{\{n\}})^T (y^{\{1\}}, \dots, y^{\{n\}}) + \frac{2}{n} (x^{\{1\}}, \dots, x^{\{n\}})^T (x^{\{1\}}, \dots, x^{\{n\}}) \theta = 0$$

Define $X = (x^{\{1\}}, \dots, x^{\{n\}})$ and $y = (y^{\{1\}}, \dots, y^{\{n\}})$

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n} X^T y + \frac{2}{n} X^T X \theta = 0$$

$$\Rightarrow \theta = (X^T X)^{-1} X^T y = X^+ y$$

X^+ is the **pseudo-inverse** of X
 $X^T X X^+ = X^T$

$$\theta = (X^T X)^{-1} X^T y = X^+ y$$

$X_{n \times d}$

$n = \text{instances}$ $d = \text{dimension}$

$$X^T X = \left[\begin{array}{c} d \times n \end{array} \right] \left[\begin{array}{c} n \times d \end{array} \right] = \left[\begin{array}{c} d \times d \end{array} \right]$$

Not a big matrix because $n \gg d$ This matrix is invertible most of the times. If we are VERY unlucky and columns of $X^T X$ are not linearly independent (it's not a full rank matrix), then it is not invertible.

LCF $\hat{y}_p = \theta_0 + \theta_1 x_1 + \dots + \theta_d x_d = X \cdot \theta \in \mathbb{R}$ $X = [1, x_1^{(1)}, \dots, x_d^{(1)}]_{1 \times d+1}$

minimize
 $\downarrow$
 $L(\theta) = E(\theta) = \frac{1}{N} \sum_{i=1}^N (y_a^{(i)} - \hat{y}_p^{(i)})^2 \leadsto \text{Convex}$

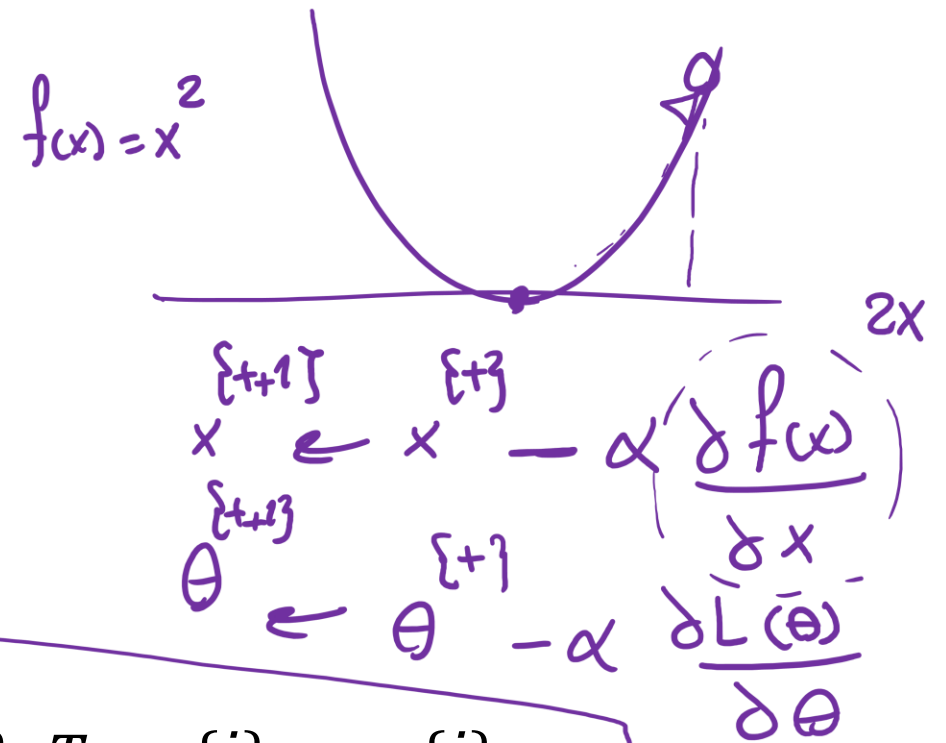
$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix}_{d+1 \times 1} \quad y = \begin{bmatrix} \end{bmatrix}_{n \times 1}$$

$$\frac{\partial L(\theta)}{\partial \theta} = 0 \Rightarrow \underbrace{\theta}_{d+1 \times 1} = \underbrace{(X^T X)^{-1}}_{d+1 \times d+1} \underbrace{X^T}_{d+1 \times n} \underbrace{y}_{n \times 1}$$

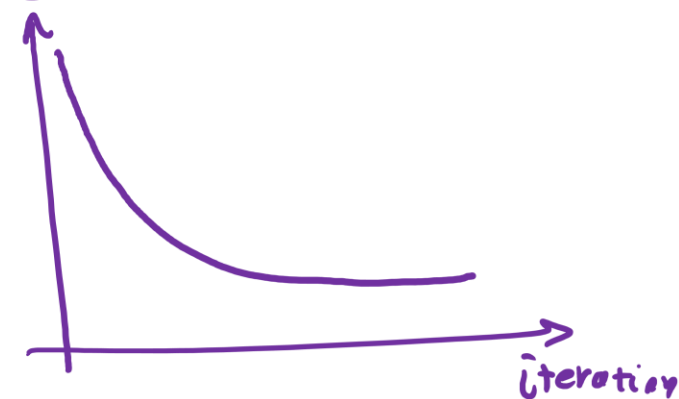
Alternative Way to Optimize

- The matrix inversion in $\hat{\theta} = (X^T X)^{-1} X^T y$ can be very expensive to compute

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n} \sum_{i=1}^N (x^{(i)})^T (y^{(i)} - x^{(i)} \theta)$$



Gradient descent



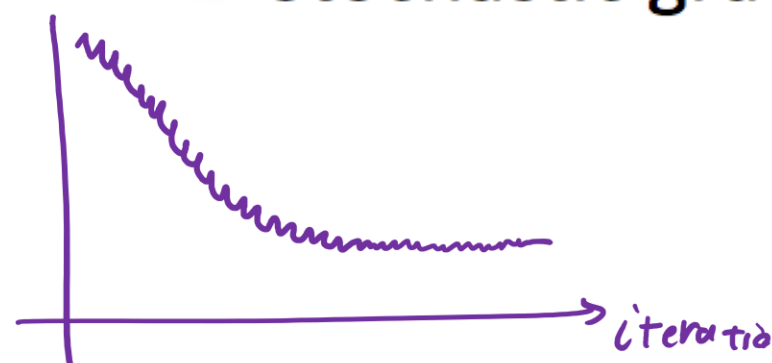
$$\hat{\theta}^{t+1} \leftarrow \hat{\theta}^t + \frac{\alpha}{n} \sum_{i=1}^N (x^{(i)})^T (y^{(i)} - x^{(i)} \theta)$$

FGD

100 $\rightarrow$ 10 ... 10 ... 10

1 epoch BGD

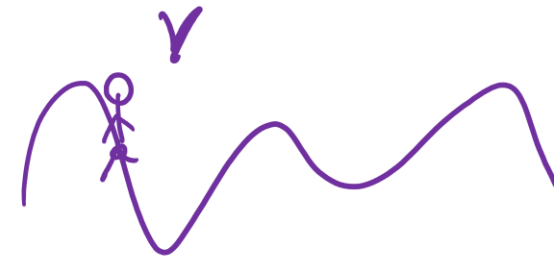
Stochastic gradient descent (use one data point at a time)



$$\hat{\theta}^{t+1} \leftarrow \hat{\theta}^t + \beta_t \times (x^{(i)})^T (y^{(i)} - x^{(i)} \theta)$$

SGD

Recap



- Stochastic gradient update rule

$$\hat{\theta}^{t+1} \leftarrow \hat{\theta}^t + \beta_t \times (x^{\{i\}})^T (y^{\{i\}} - x^{\{i\}} \theta)$$

SGD

- Pros: on-line, low per-step cost
- Cons: coordinate, maybe slow-converging

- Gradient descent

$$\hat{\theta}^{t+1} \leftarrow \hat{\theta}^t + \frac{\alpha}{n} \sum_{i=1}^N (x^{\{i\}})^T (y^{\{i\}} - x^{\{i\}} \theta) \rightsquigarrow$$

BGD

FGD

- Pros: fast-converging, easy to implement
- Cons: need to read all data

- Solve normal equations

$$\theta = (X^T X)^{-1} X^T y \rightsquigarrow \text{closed form}$$

- Pros: a single-shot algorithm! Easiest to implement.
- Cons: need to compute inverse $(X^T X)^{-1}$, expensive, numerical issues (e.g., matrix is singular ..)

Linear regression for classification

Raw Input $x = (1, x_1, \dots, x_{256})$

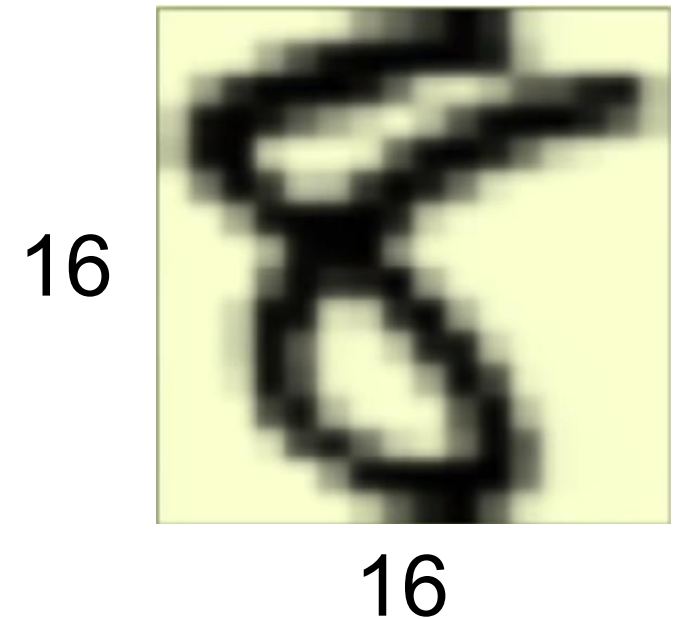
Linear model $(\theta_0, \theta_1, \dots, \theta_{256})$

Extract useful information

intensity and symmetry $x = (1, x_1, x_2)$

Sum up all the pixels = intensity

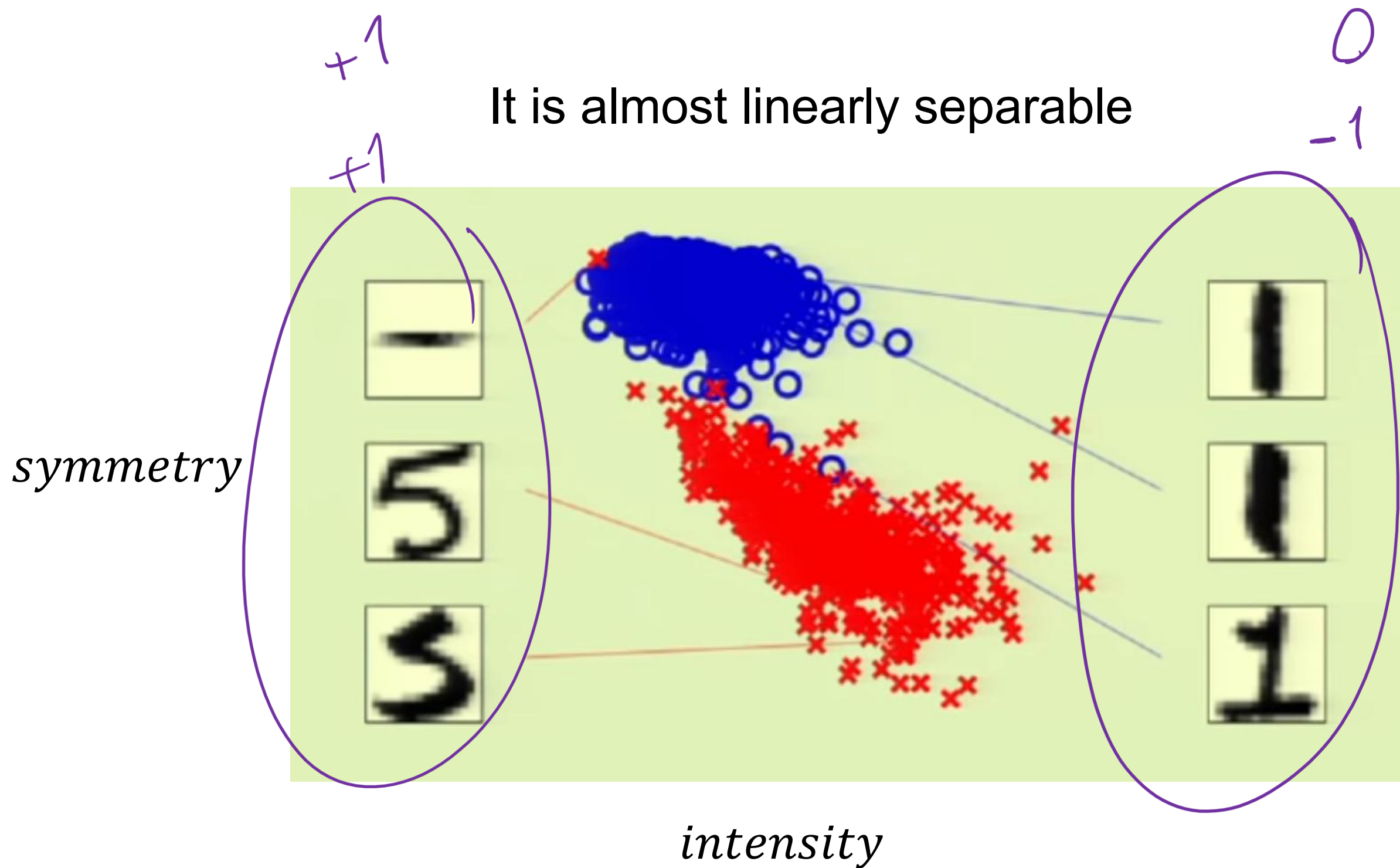
Symmetry = -(difference between flip version)



$$x = (1, x_1, x_2)$$

$$x_1 = \text{intensity} \quad x_2 = \text{symmetry}$$

It is almost linearly separable



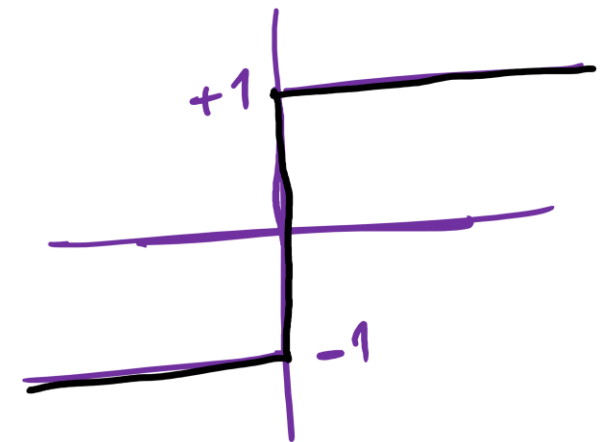
Linear regression for classification

Binary-valued functions are also real-valued $\pm 1 \in \mathbb{R}$

Use linear regression $\hat{y}_p \in \mathbb{R}$
 $x^{(i)} \theta \approx y^{(i)} = \pm 1$ $i = \text{index of a data-point}$

$$x^{(i)} \theta = 0 \quad x \perp \theta$$

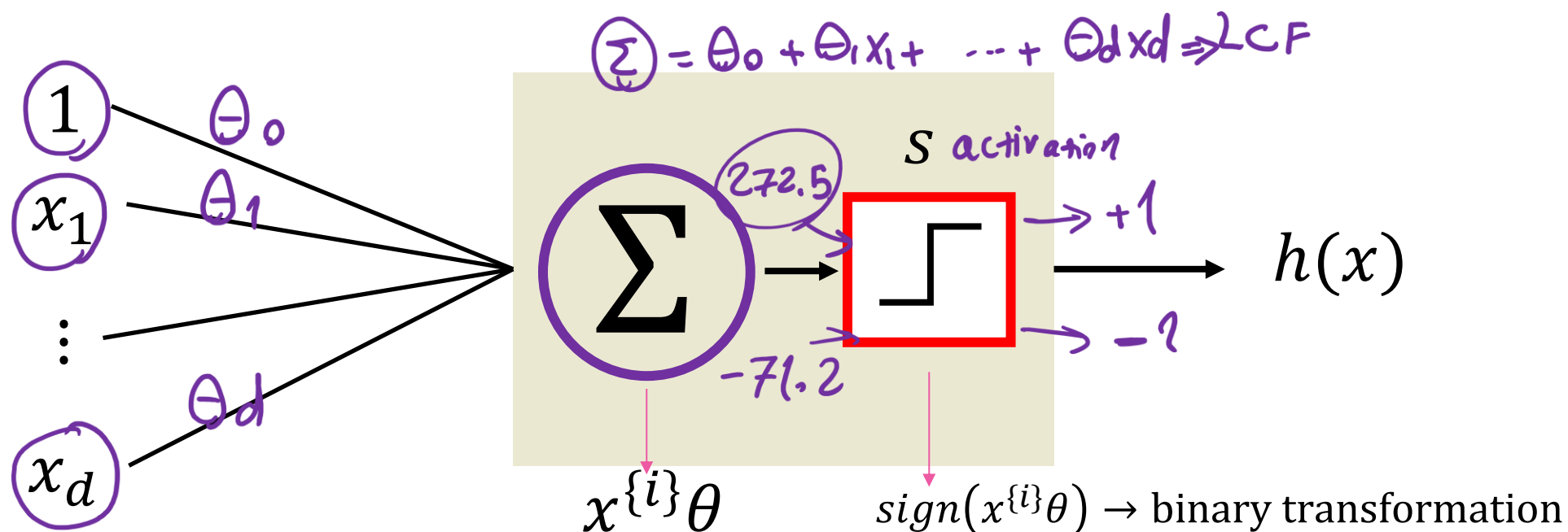
Let's calculate, $\text{sign}(x^{(i)} \theta) = \begin{cases} -1 & x^{(i)} \theta < 0 \\ 0 & x^{(i)} \theta = 0 \\ 1 & x^{(i)} \theta > 0 \end{cases}$

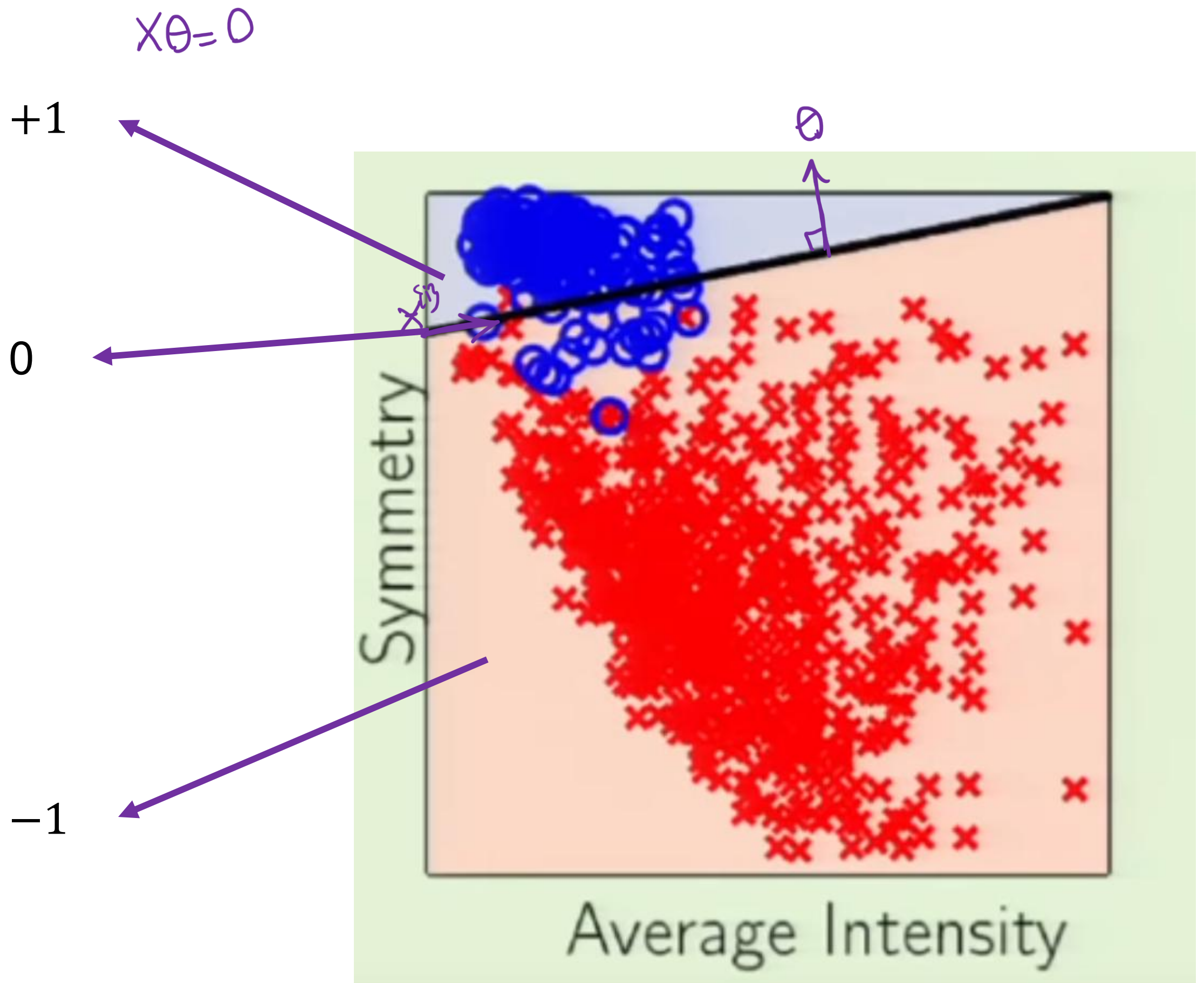


$x^{(i)}$

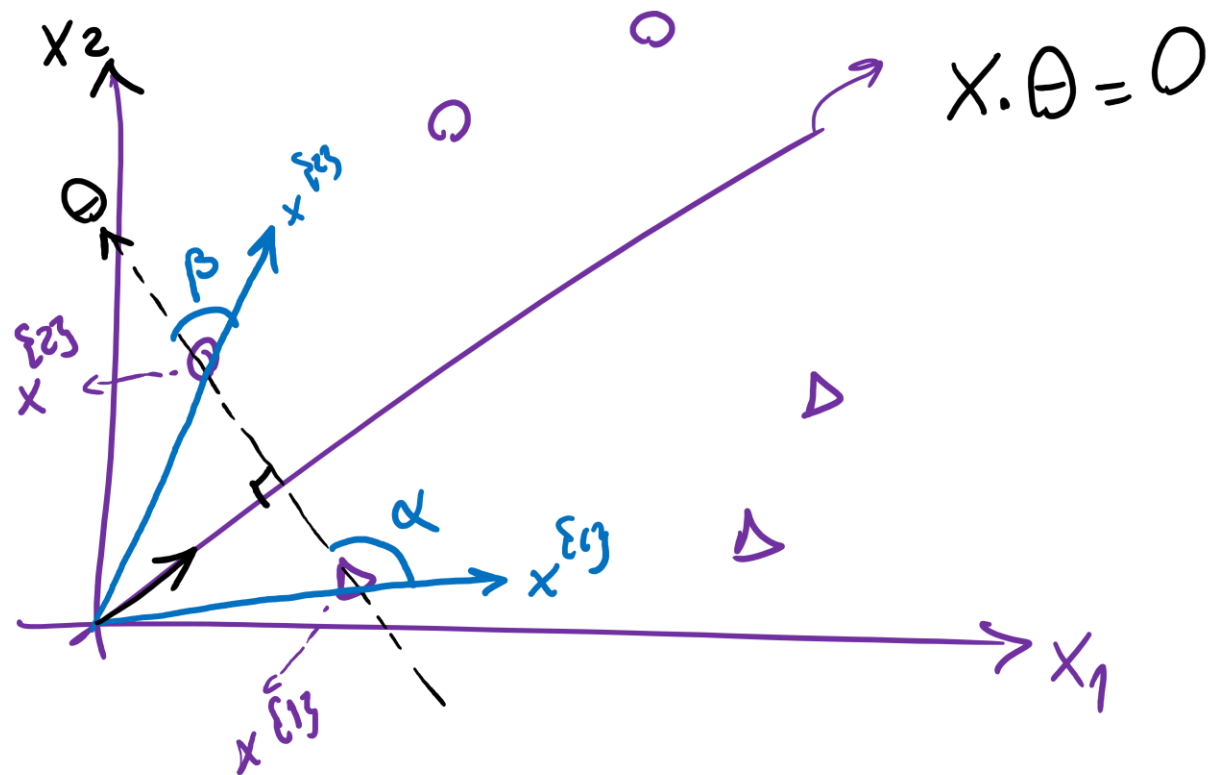
For one data point (data-point i) with d dimensions (instance):

a learning block





Not really the best for classification, but t's a good start

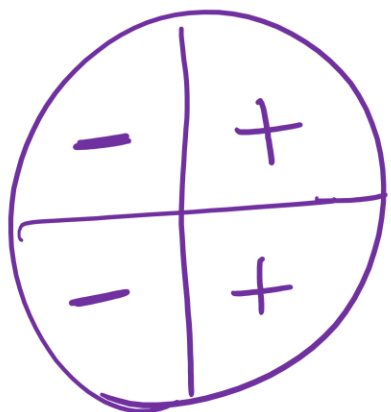


$$x^{(1)} \cdot \Theta = |x^{(1)}| \cdot |\Theta| \cdot \underbrace{\cos \alpha}_{-} > 0$$

$$\text{Sign}(-) = -1$$

$$x^{(2)} \cdot \Theta = |x^{(2)}| \cdot |\Theta| \underbrace{\cos \beta}_{+} < 0$$

$$\text{Sign}(+) = +1$$

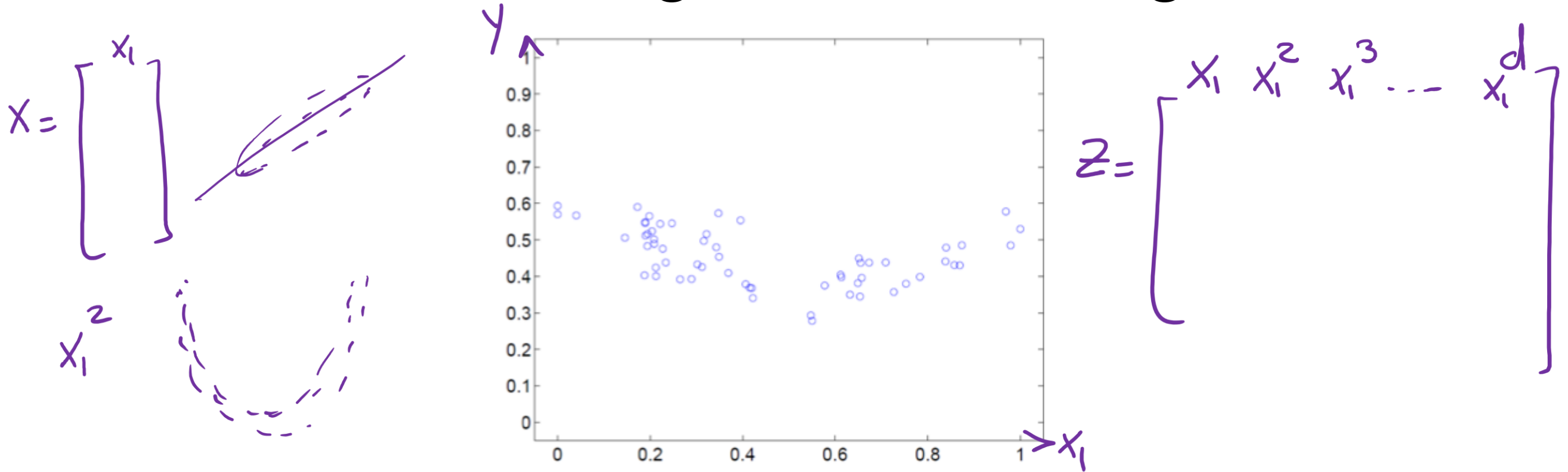


$$\hat{y}_p = \Theta_0 + \Theta_1 x_1 + \Theta_2 x_2$$

Outline

- Supervised Learning
- Linear Regression
- Extension 

Extension to Higher-Order Regression



- Want to fit a polynomial regression model

$$y = \theta_0 + \theta_1 \underline{x} + \theta_2 \underline{x}^2 + \dots + \theta_d \underline{x}^d + \epsilon$$

- $z = \{1, x, x^2, \dots, x^d\} \in R^d$ and $\theta = (\theta_0, \theta_1, \theta_2, \dots, \theta_d)^T$

$$y = z\theta$$

Least Mean Square Still Works the Same

- Given n data points, find θ that minimizes the mean square error

$$\hat{\theta} = \operatorname{argmin}_{\theta} L(\theta) = \frac{1}{n} \sum_{i=1}^N (y^{\{i\}} - z^{\{i\}} \theta)^2$$

- Our usual trick: set gradient to 0 and find parameter

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n} \sum_{i=1}^N (z^{\{i\}})^T (y^{\{i\}} - z^{\{i\}} \theta) = 0$$

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n} \sum_{i=1}^N (z^{\{i\}})^T y^{\{i\}} + \frac{2}{n} \sum_{i=1}^N (z^{\{i\}})^T z^{\{i\}} \theta = 0$$

Matrix Version of the Gradient

$$z = \{1, x, x^2, \dots, x^d\} \in R^d \quad y = \{y^{\{1\}}, y^{\{2\}}, \dots, y^{\{n\}}\}$$

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n} z^T y + \frac{2}{n} z^T z \theta = 0$$

$$\Rightarrow \theta = (z^T z)^{-1} z^T y = z^+ y$$

- If we choose a different maximal degree **d** for the polynomial, the solution will be different.

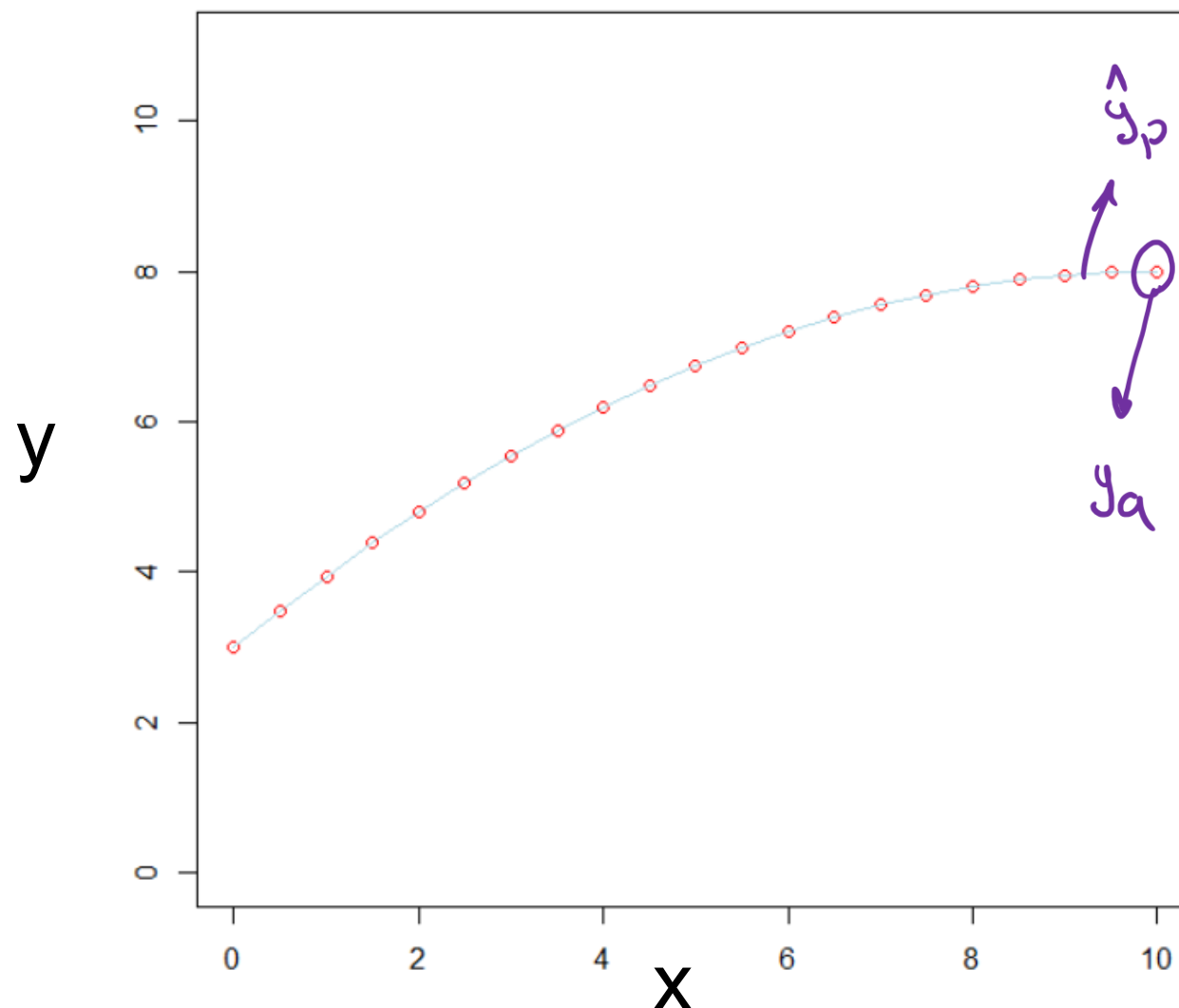
What is happening in polynomial regression?

$$x = [0, 0.5, 1, \dots, 9.5, 10]$$

$$y = [3, 3.4875, 3.95, \dots, 7.98, 8]$$

$$f = \theta_0 + \theta_1 x + \theta_2 x^2$$

$$\theta_0 = 3; \theta_1 = 1; \theta_2 = -0.5$$

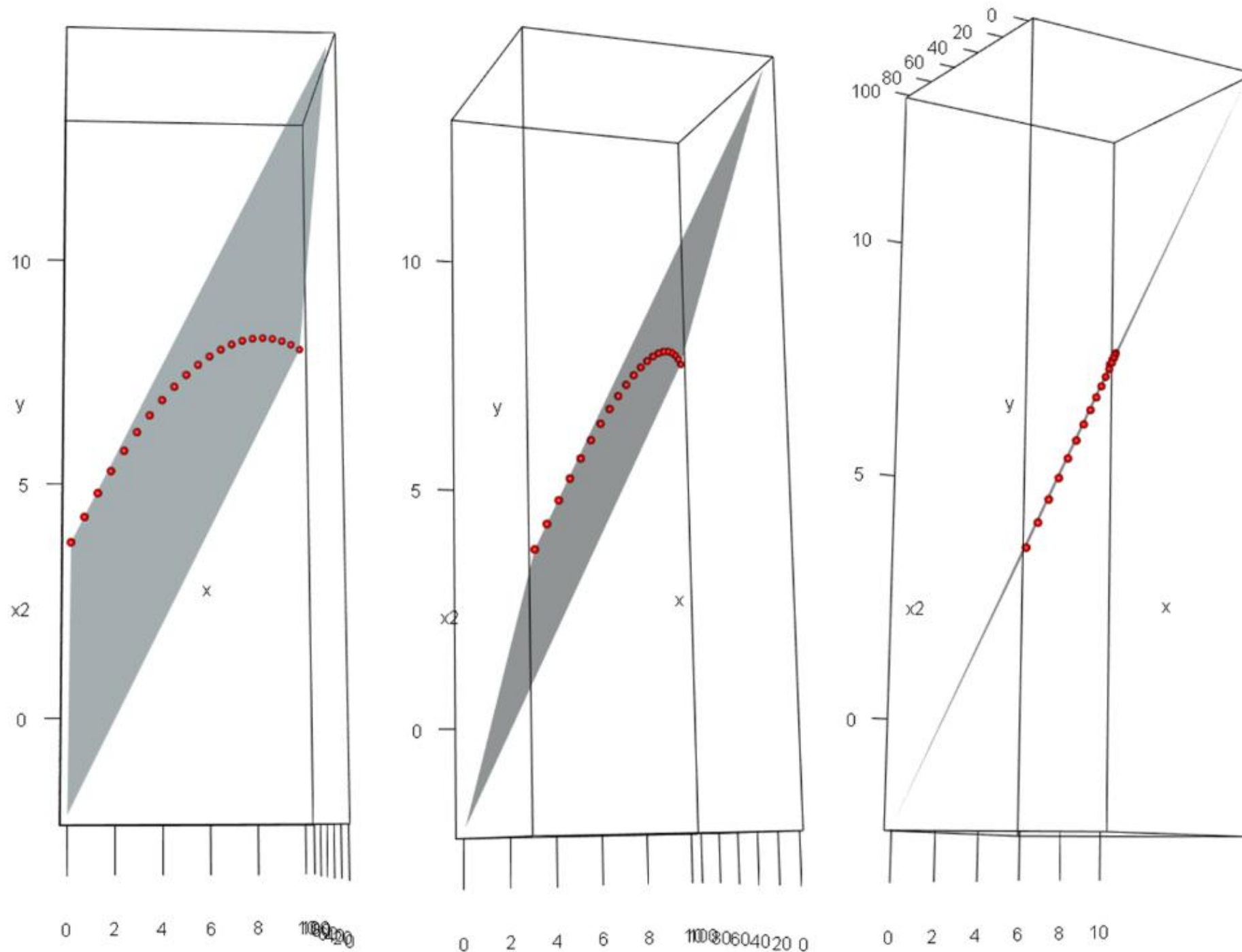


$$\text{RMSE}=0$$

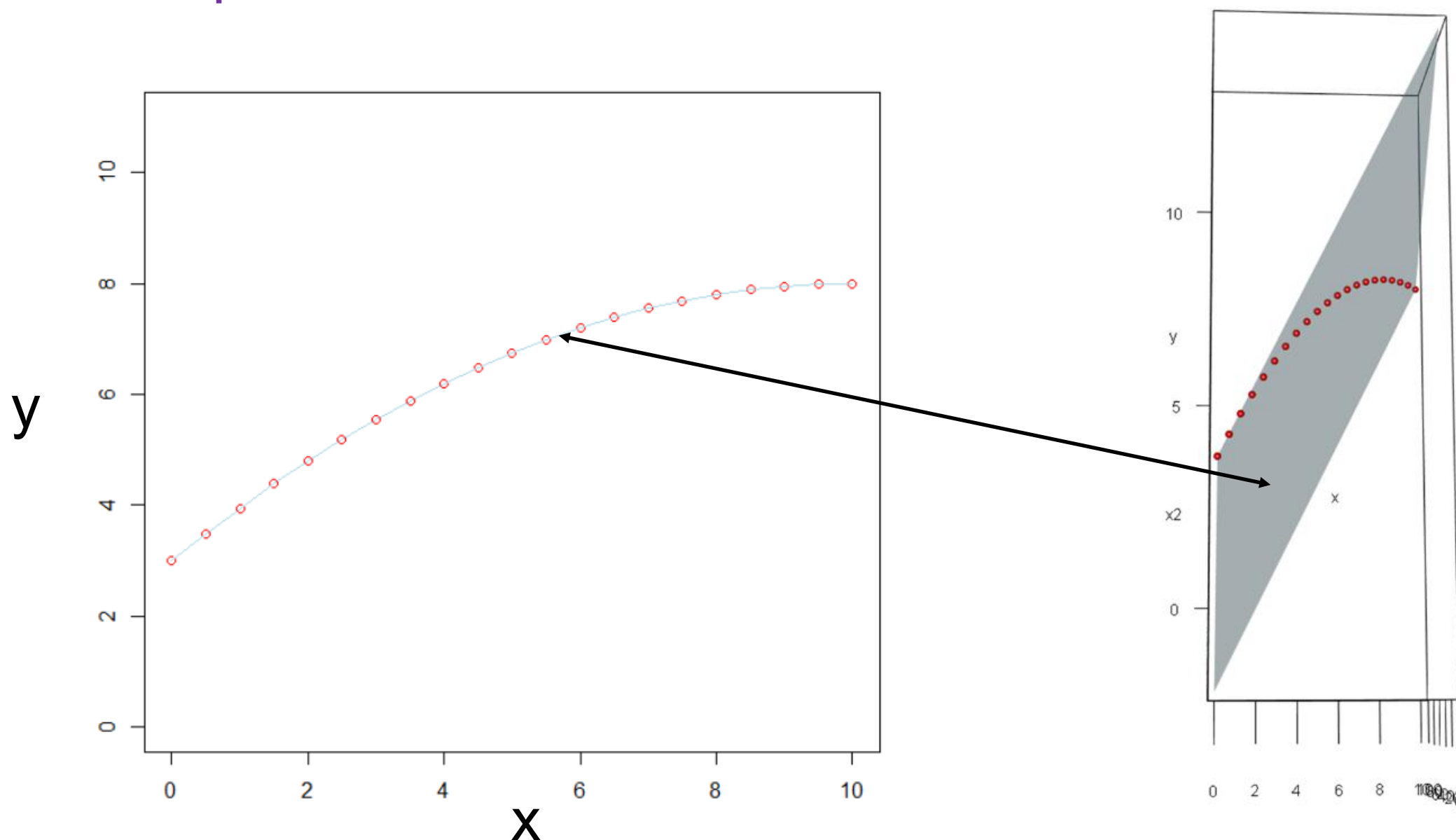
$$\text{MSE}=0$$

Let's add to the feature space

$$x_1 = [0, 0.5, 1, \dots, 9.5, 10] \quad x_2^2 = [0, 0.25, 1, \dots, 90.25, 100]$$
$$y = [3, 3.4875, 3.95, \dots, 7.98, 8]$$



We are fitting a D -dimensional hyperplane in a $D+1$ dimensional hyperspace (in above example a 2D plane in a 3D space). That hyperplane really is 'flat' / 'linear' in 3D. It can be seen a non-linear regression (a curvy line) in our 2D example in fact it is a flat surface in 3D. So the fact that it is mentioned that the model is linear in parameters, it is shown here.

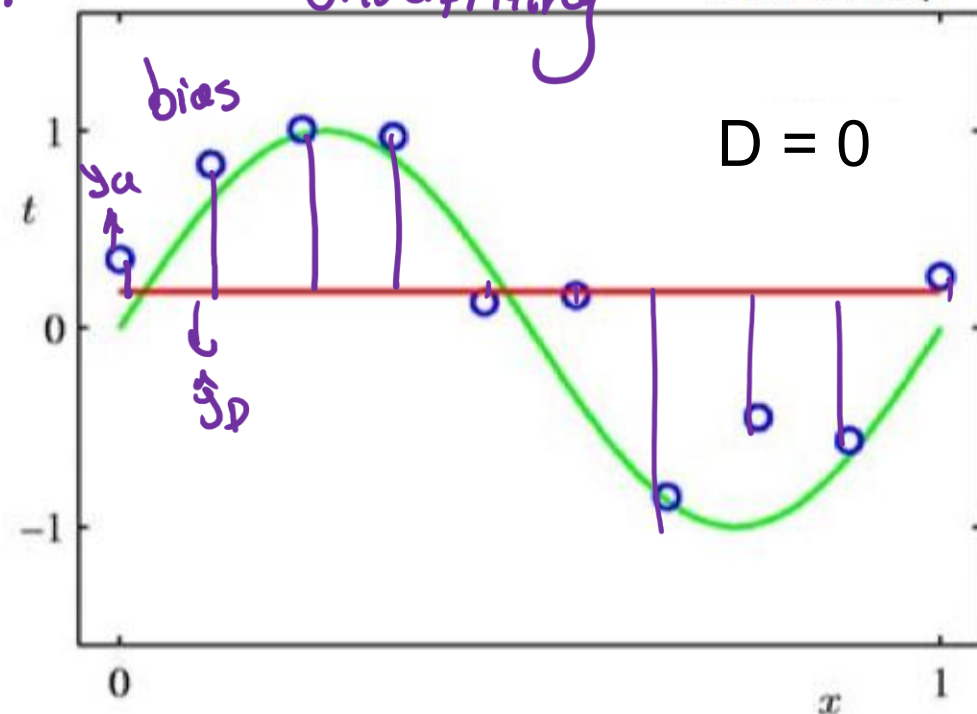


Increasing the Maximal Degree

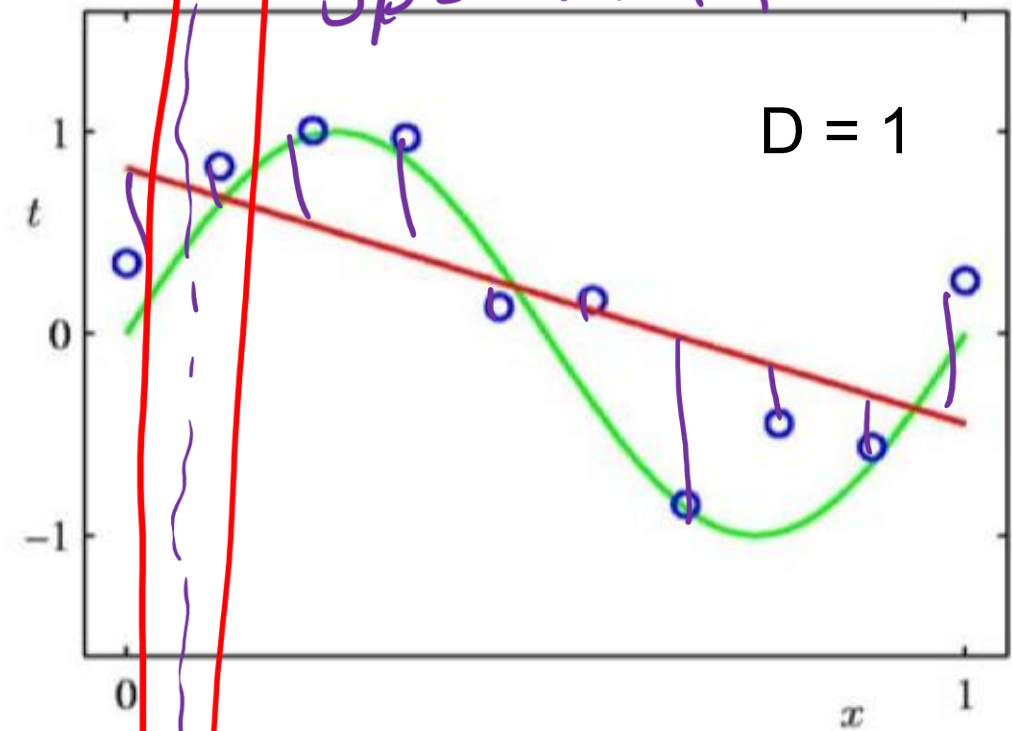
$$\hat{y}_p = \theta_0$$

underfitting

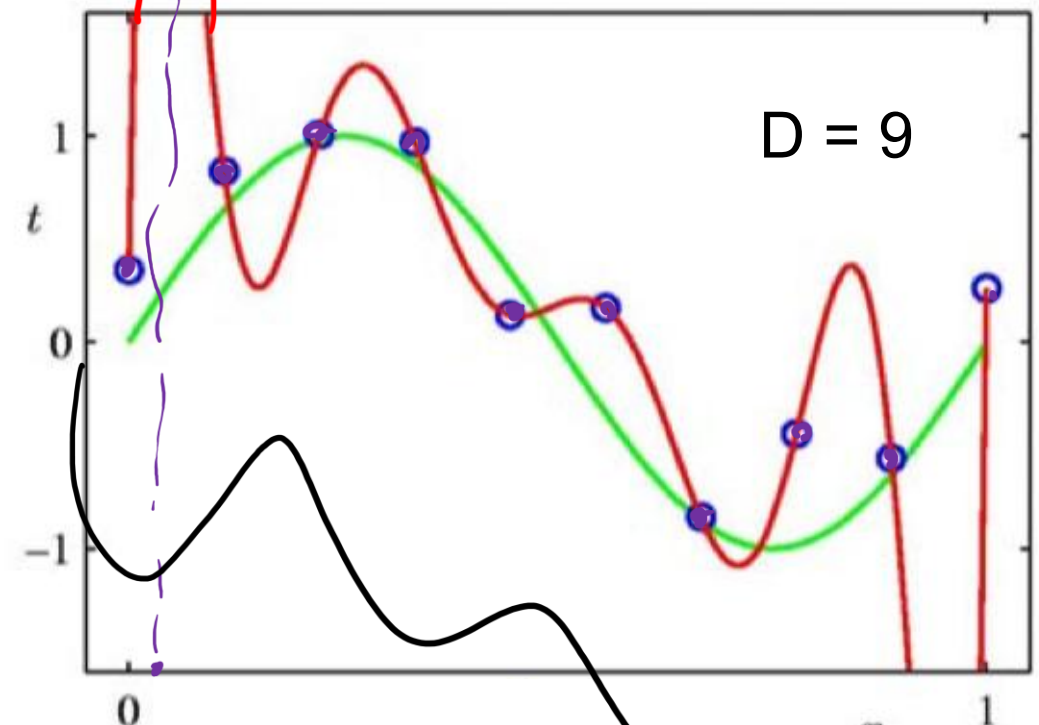
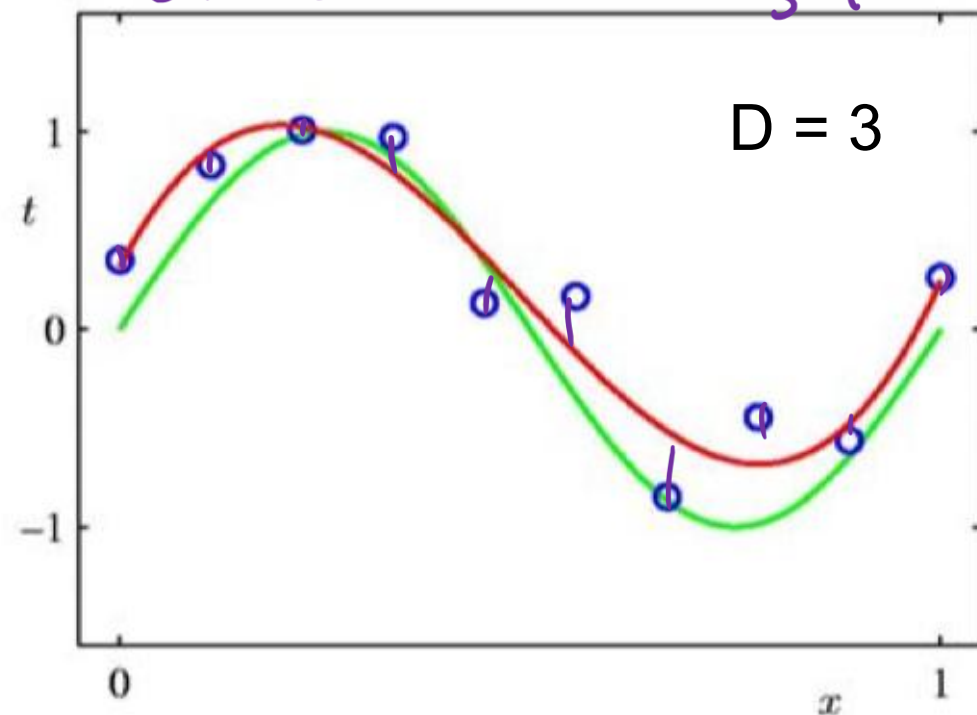
from Bishop



$$\hat{y}_p = \theta_0 + \theta_1 x_1$$



$$\hat{y}_p = \theta_0 + \theta_1 x_1 + \dots + \theta_3 x_1^3$$



$$X = \begin{bmatrix} & & \\ & & \\ & & \end{bmatrix}_{100 \times d}$$

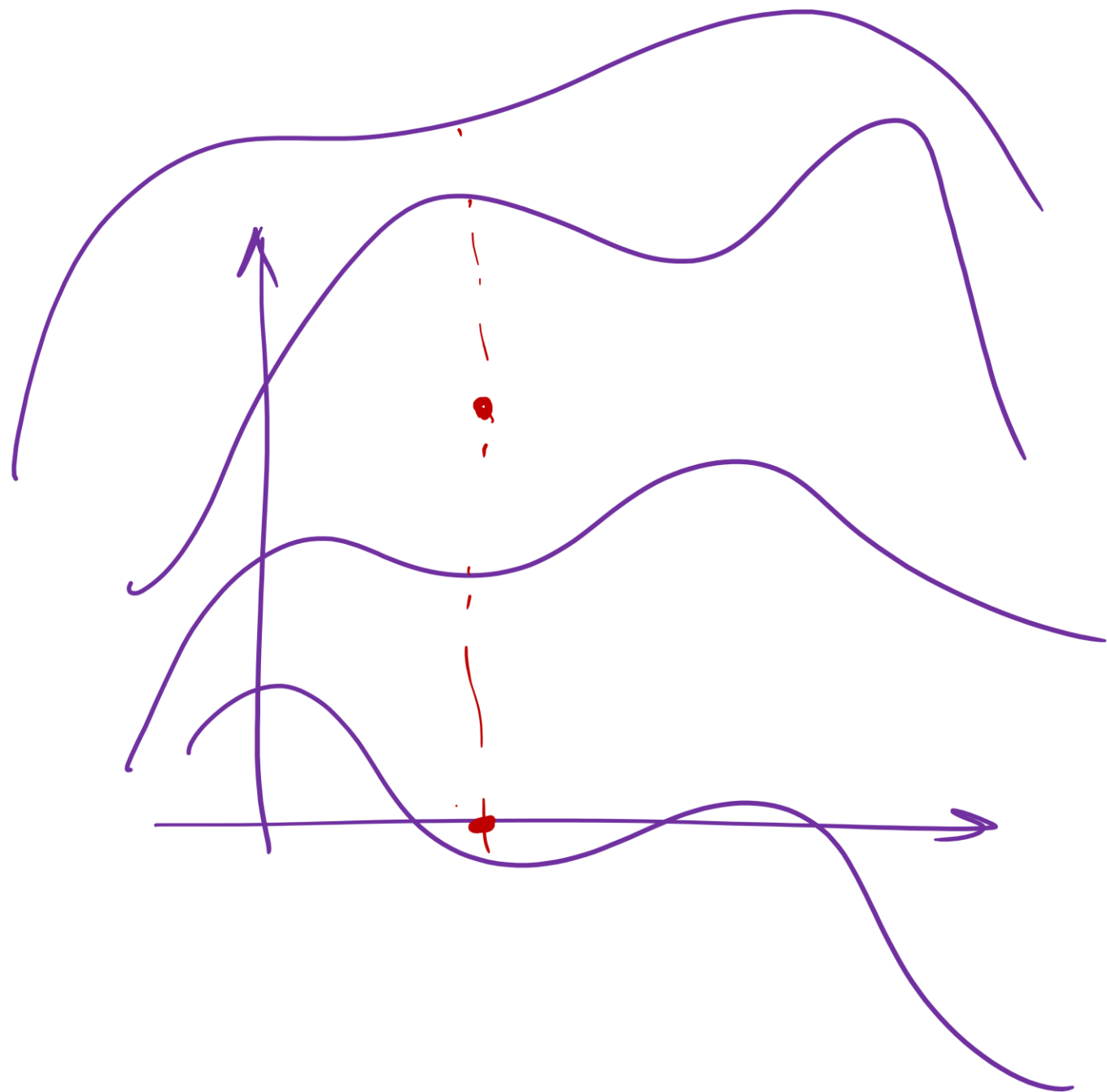
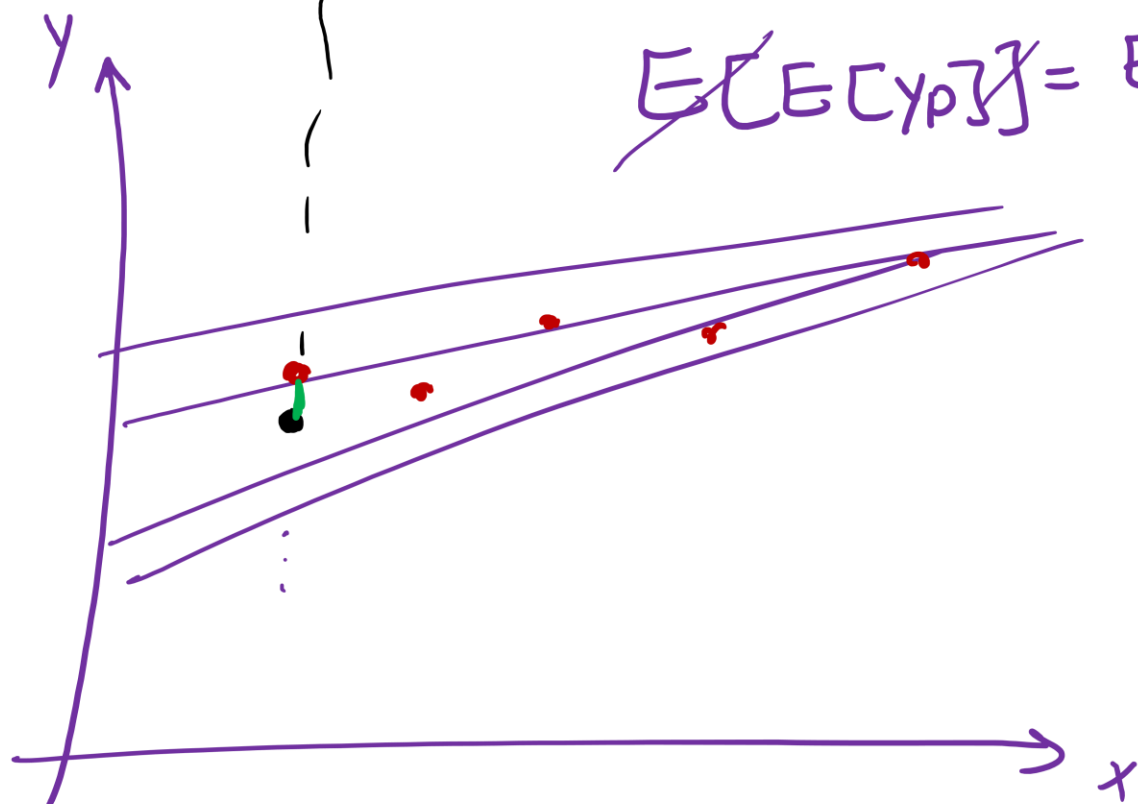
$$X_{(1)} = \begin{bmatrix} & & \end{bmatrix}_{10 \times d} \dots$$

$$X_{(10)} = \begin{bmatrix} & & \end{bmatrix}_{10 \times d}$$

GIR

$$E[\hat{y}_p] = \frac{1}{N} \sum (\hat{y}_p)$$

$$E[E[y_p]] = E[y_p]$$



$$\text{avg}(\cdot) = E[\cdot]$$

Bias-Variance Trade off

[Animation](#)

We will have multiple prediction values (i.e. through Cross validation) $E[y_p]$

$$L(\theta) = \frac{1}{n} \sum_{i=1}^N (y^{\{i\}} - x^{\{i\}} \theta)^2 = E[(y_a - y_p)^2] = \text{bias}^2 + \text{Variance}$$

$$(y_a - y_p)^2 = \underbrace{(y_a - E[y_p])^2}_a + \underbrace{(E[y_p] - y_p)^2}_b$$

$$E[(y_a - E[y_p])^2 + (E[y_p] - y_p)^2 + 2(y_a - E[y_p])(E[y_p] - y_p)]$$

$$E[(y_a - E[y_p])^2] + E[(E[y_p] - y_p)^2]$$

$$(y_a - E[y_p])^2 + E[(E[y_p] - y_p)^2] + 0 = \text{bias} + \text{Variance}$$

Bias-Variance Trade off

[Animation](#)

We will have multiple prediction values (i.e. through Cross validation) $E[y_p]$

$$L(\theta) = \frac{1}{n} \sum_{i=1}^N (y^{\{i\}} - x^{\{i\}} \theta)^2 = E \left[(y_a - y_p)^2 \right]$$

$$(y_a - y_p)^2 = (y_a - E[y_p] + E[y_p] - y_p)^2$$

$$= (y_a - E[y_p])^2 + (E[y_p] - y_p)^2 + 2(y_a - E[y_p])(E[y_p] - y_p)$$

$$E \left[(y_a - y_p)^2 \right] = (y_a - E[y_p])^2 + E \left[(E[y_p] - y_p)^2 \right]$$

$$= [Bias]^2 + Variance$$

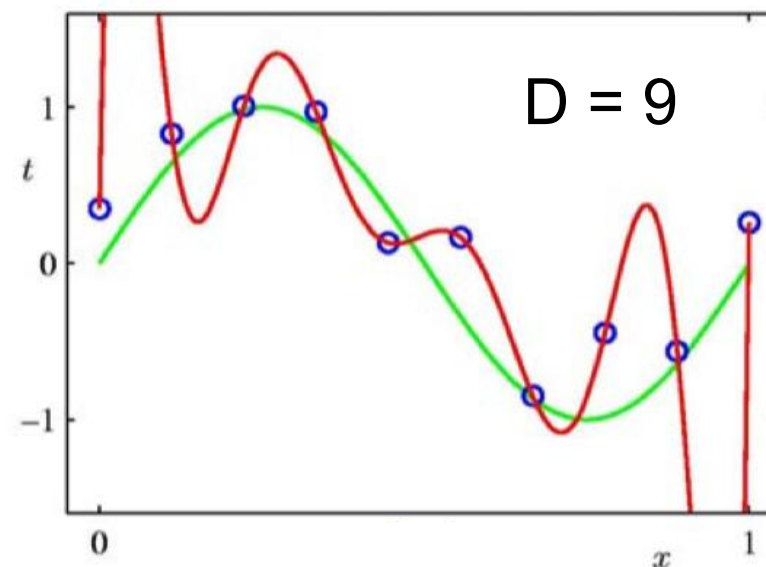
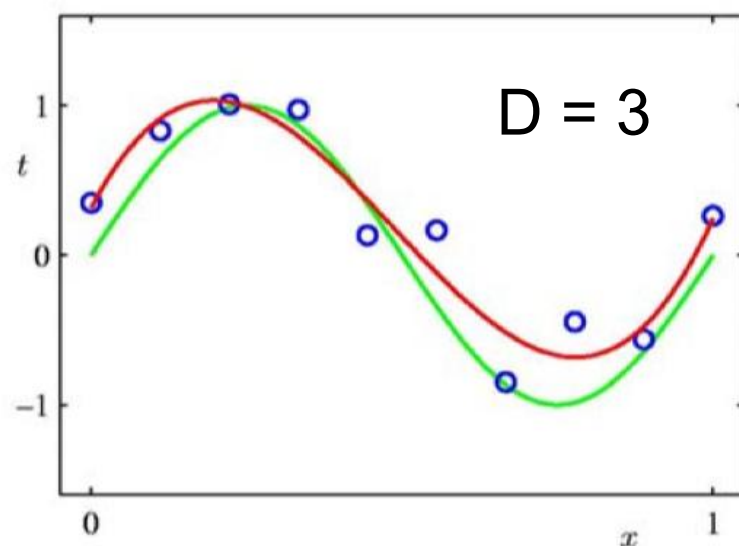
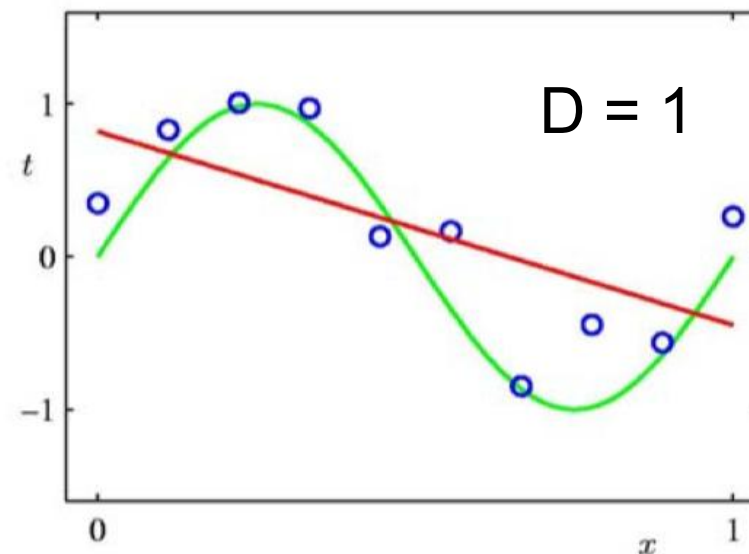
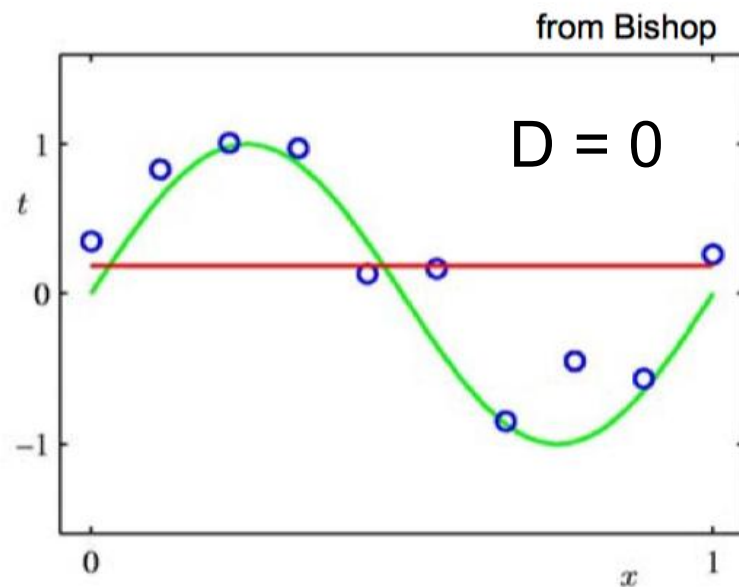
$$= [true\ value - mean(predictions)]^2 - mean[(mean(prediction) - prediction)^2]$$

Why $E[2(y_a - E[y_p])(E[y_p] - y_p)] = 0$?

$y_a - E[y_p]$ is a scalar, therefore $E[y_a - E[y_p]] = y_a - E[y_p]$

$$\begin{aligned} & E[2(y_a - E[y_p])(E[y_p] - y_p)] \\ &= 2(y_a - E[y_p])E[E[y_p] - y_p] \\ &= 2(y_a - E[y_p])\left(E[E[y_p]] - E[y_p]\right) \\ &= 2(y_a - E[y_p])(E[y_p] - E[y_p]) = 0 \end{aligned}$$

Which One is Better?



- Can we increase the maximal polynomial degree to very large, such that the curve passes through all training points?
 - We will know the answer in next lecture.

Take-Home Messages

- Supervised learning paradigm
- Linear regression and least mean square
- Extension to high-order polynomials